Федеральное государственное автономное образовательное учреждение высшего образования
Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина «*Тестирование программного обеспечения*»

Отчёт

По лабораторной работе №1

Вариант: 408138

Студент:

Ильин Н. С.

Алхимовици А.

группа *P3310*

Преподаватель:

Кулинич Я. В.

г. Санкт-Петербург, 2026 г.

Описание задания

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

Вариант **408138**

1. Функция **sec(x)**
2. Программный модуль для работы с **кучей** (http://www.cs.usfca.edu/~galles/visualization/HeapSort.html)
3. Описание предметной области:

Артур неожиданно почувствовал, как на его бесплотной голове зашевелились волосы, когда он начал медленно, но неуклонно двигаться к пульту, но это был всего лишь, как он тут же догадался, наезд камеры при съемке.

Выполнение

Исходный код

https://github.com/senya-2011/Software-testing-course

Задание 1: *sec(x)*

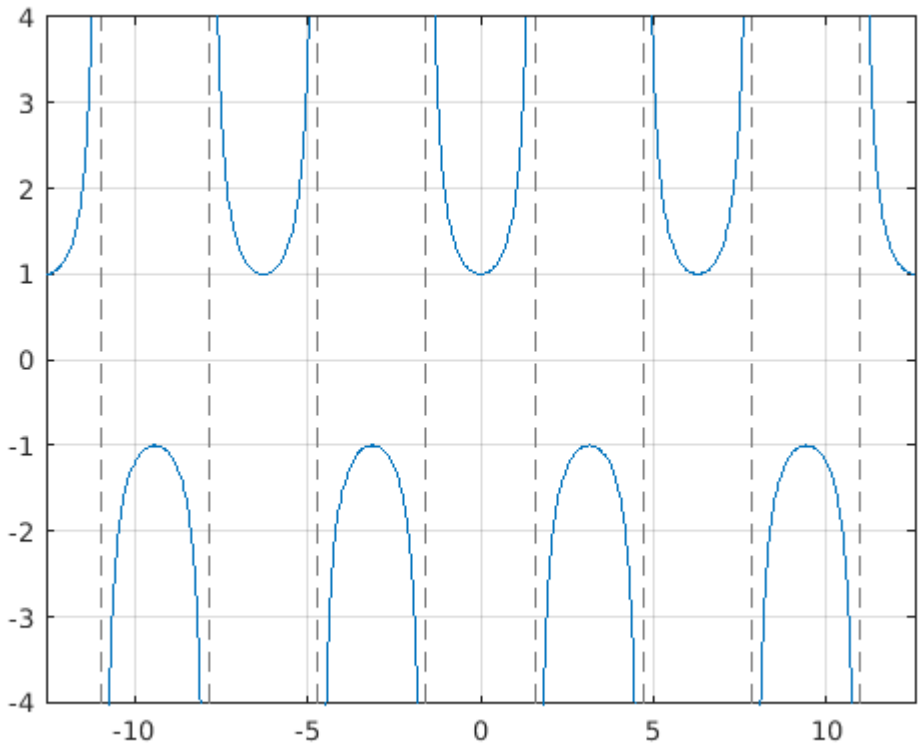


График функции sec(x) с табличными значениями в точках.

Отчет о тестировании функции sec (x)

1. Выбор характеристических точек для тестирования:

Для обеспечения репрезентативного покрытия области определения и области значений функции $y = \sec(x)$ были выбраны следующие классы тестовых данных:

- $0, \pm\pi, \pm 2\pi$ — узловые точки, где $\cos(x) = \pm 1$. В этих точках значение функции равно ± 1 (экстремумы), что позволяет проверить базовую корректность алгоритма без влияния погрешности деления на малые числа.
- $\pm\frac{\pi}{3}$ — стандартные табличные значения, где $\cos(x) = 0.5$, следовательно, $\sec(x) = 2.0$. Используются для проверки точности вычислений на целочисленных результатах.
- $\pm\frac{\pi}{2}, \pm\frac{3\pi}{2}$ — точки разрыва второго рода (вертикальные асимптоты). В этих точках функция не определена ($\lim_{x\to\pi/2} \sec(x) = \infty$). Реализация должна корректно идентифицировать выход за пределы

вычислительных возможностей double и возвращать Infinity, а не ошибочное конечное число.

- Double.NaN — проверка на проброс нечислового значения (должно возвращаться NaN согласно стандарту IEEE 754).
- Double.POSITIVE_INFINITY — проверка поведения при бесконечном аргументе (результат NaN, так как предел не существует).

2. Верификация математических инвариантов (свойств функции):

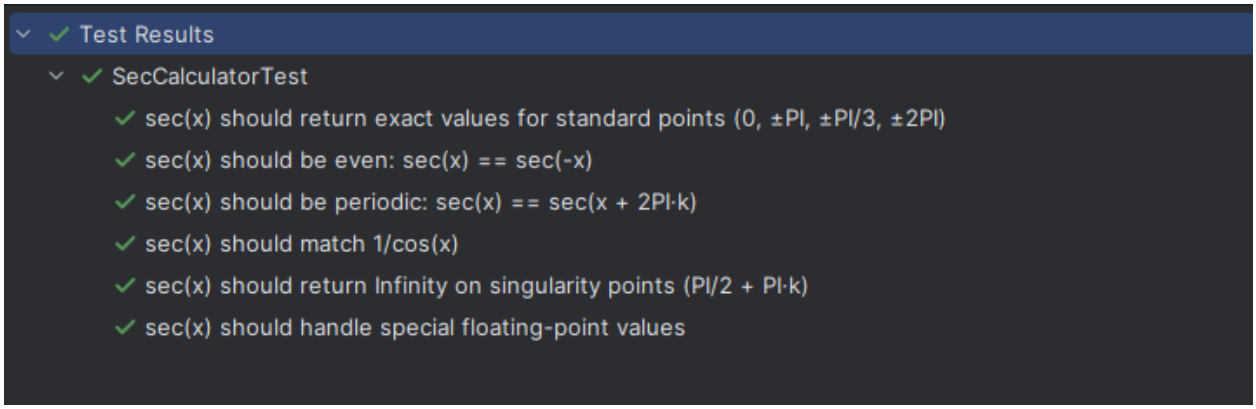
Вместо перебора дискретных точек в промежутках использовалась проверка фундаментальных свойств функции на непрерывных диапазонах:

- **Четность:** Проверка условия $\sec(x) = \sec(-x)$ на случайных значениях в диапазоне $[0, 1.5]$. Это гарантирует симметричность реализации относительно оси ординат.
- **Периодичность:** Проверка условия $\sec(x) = \sec(x + 2\pi k)$ для $k \in \mathbb{Z}$. Это подтверждает корректность работы алгоритма на всей числовой оси путем сведения задачи к вычислению на базовом периоде.

Итоговое тестовое покрытие:

Разработанный набор тестов обеспечивает полное покрытие функциональности класса SecCalculator:

1. Подтверждена корректность вычислений в области сходимости ряда Тейлора (совпадение с эталонной реализацией).
2. Доказано выполнение математических свойств функции (четность, периодичность), что позволяет экстраполировать корректность работы на всю числовую ось.
3. Верифицировано поведение в граничных точках (асимптотах) и при передаче некорректных данных (NaN), что гарантирует устойчивость алгоритма к исключительным ситуациям.



Тесты успешно проходятся.

Задание 2: *Heap Sort*

Эталонная реализация по варианту:
http://www.cs.usfca.edu/~galles/visualization/HeapSort.html

Подход к тестированию

Тестирование проводилось с использованием white-box подхода.

Покрытие кода (Code Coverage)

Были выделены характерные точки алгоритма:

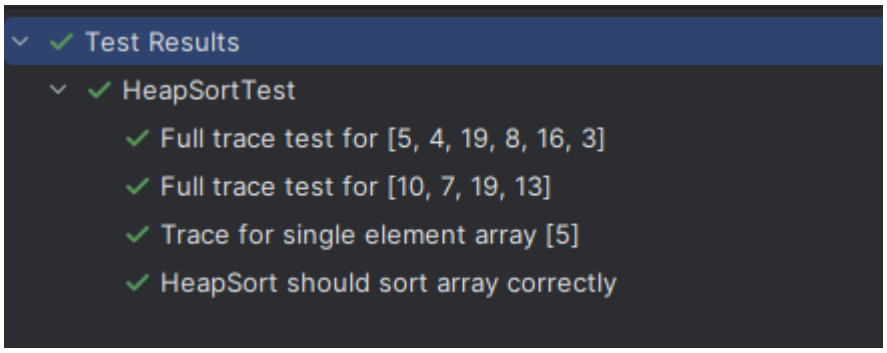
- T1 — старт сортировки
- T2 — построение кучи
- T3–T7 — проверки потомков и перестановки
- T8–T10 — этап извлечения максимума
- T11 — завершение сортировки

Для проверки покрытия была реализована трассировка выполнения (trace). Тесты сравнивают фактическую последовательность переходов с эталонной.

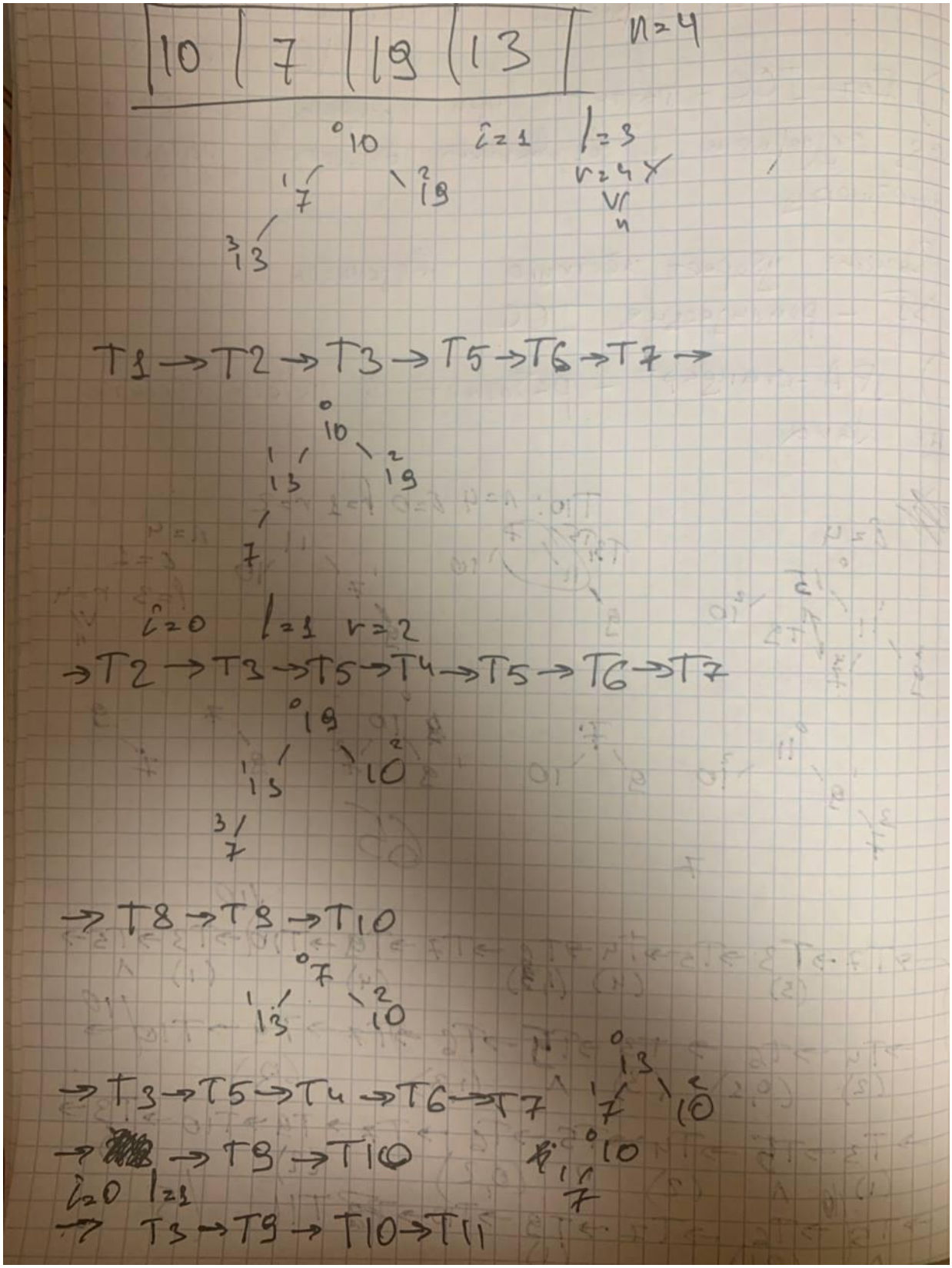
Тестирование трассировки

Для входного массива: [10, 7, 19, 13] была получена эталонная последовательность переходов.

Тест сравнивает фактическую последовательность выполнения с эталонной, что подтверждает корректность прохождения всех ветвей в алгоритме.



Тесты успешно проходятся.



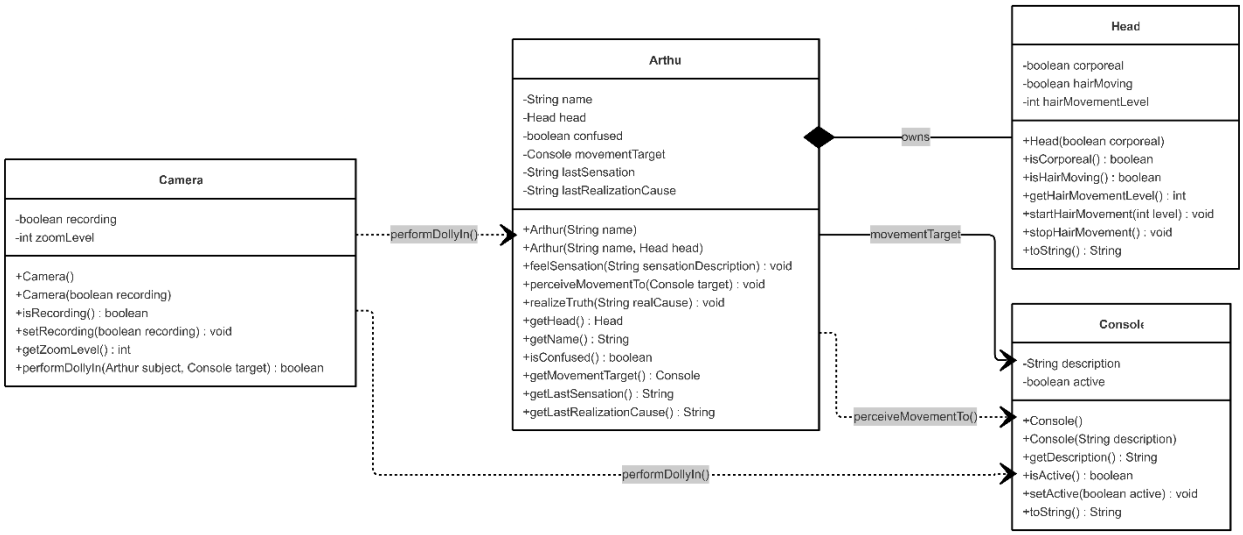
Эталонная последовательность для [10, 7, 19, 13].

Результаты тестирования:

Алгоритм Heap Sort реализован корректно и проходит все тестовые сценарии.

Задание 3: *Описание предметной области*

Спроектируем нашу доменную область в соответствии с заданным текстом.



Спроектированная UML диаграмма предметной области.

Предметная область

Предметная область task3 включает:

- Персонажа: Arthur.
- Составную часть персонажа: Head (бесплотная/обычная голова, состояние волос).
- Объекты сцены: Console (пульт) и Camera (камера).
- Действия:
  - почувствовать ощущение;
  - начать/остановить шевеление волос;
  - воспринять движение к пульту;
  - осознать истинную причину происходящего;
  - выполнить наезд камеры.
- Сценарий: Camera.performDollyIn(Arthur subject, Console target).

Цель тестирования

В соответствии с TDD необходимо убедиться, что:

- Все методы и сценарии выполняются корректно.
- Состояния персонажа и объектов изменяются в соответствии с логикой сцены.

- Граничные случаи (неактивный пульт, выключенная запись, null-аргументы, невалидные значения) обрабатываются корректно.

Набор тестов

Класс Arthur

- testArthurDefaultConstructorState
  Проверяет корректность создания Arthur(name) и начальное состояние полей.
- testArthurConstructorWithHead
  Проверяет, что Arthur(name, head) сохраняет ссылку на переданный Head.
- testArthurPreservesGeneratedName
  Проверяет, что имя персонажа сохраняется для произвольных строк.
- testArthurConstructorNullArgs
  Проверяет выброс NullPointerException при null в конструкторах.
- testArthurMethodsNullInputs
  Проверяет выброс NullPointerException в методах feelSensation, perceiveMovementTo, realizeTruth.

Класс Head

- testHeadCreationAndDefaults
  Проверяет сохранение признака corporeal и значения по умолчанию.
- testStartHairMovementPositiveLevel
  Проверяет запуск шевеления волос при положительном уровне.
- testStartHairMovementInvalidLevel
  Проверяет выброс IllegalArgumentException при level <= 0.
- testStopHairMovement
  Проверяет остановку шевеления волос и сброс уровня.

Класс Console

- testConsoleDefaultState
  Проверяет состояние по умолчанию ("Пульт", active = true).
- testConsoleCustomDescription
  Проверяет сохранение пользовательского описания и корректность toString().
- testConsoleNullDescription
  Проверяет выброс NullPointerException при null-описании.

- testConsoleSetActive
  Проверяет корректное изменение флага активности.

Класс Camera

- testCameraDefaultState
  Проверяет состояние по умолчанию (recording = true, zoomLevel = 1).
- testCameraConstructorRecordingFlag
  Проверяет сохранение флага записи из конструктора.
- testCameraSetRecording
  Проверяет корректное изменение флага recording.
- testPerformDollyInSuccess
  Проверяет успешное выполнение сценария и все связанные изменения состояния.
- testPerformDollyInWhenNotRecording
  Проверяет, что при recording = false сценарий не выполняется, метод возвращает false.
- testPerformDollyInWhenConsoleInactive
  Проверяет, что при неактивном пульте сценарий не выполняется.
- testPerformDollyInZoomIncrement
  Проверяет, что zoomLevel увеличивается на 1 при каждом успешном вызове.
- testPerformDollyInNullArgs
  Проверяет выброс NullPointerException при null-субъекте или null-цели.

Реализация по TDD

После написания тестов реализуются классы Arthur, Head, Console, Camera и сценарная логика performDollyIn так, чтобы пройти все тесты и зафиксировать ожидаемое поведение доменной модели.

Test Results	659 ms
DomainModelStateTest	659 ms
Given: Entity Arthur	206 ms
When: created with a name only	29 ms
When: created with a specific Head	3 ms
When: created with arbitrary names	165 ms
When: initialized with null arguments	5 ms
When: methods are called with null arguments	4 ms
Given: Entity Head	101 ms
When: created with arbitrary corporeal flag	49 ms
When: startHairMovement is called with positive level	45 ms
When: startHairMovement is called with invalid level	3 ms
When: stopHairMovement is called	4 ms
Given: Entity Console	82 ms
When: created with defaults	4 ms
When: created with arbitrary description	49 ms
When: created with null description	2 ms
When: setActive is called	27 ms
Given: Entity Camera	41 ms
Given: A scene with Arthur, Console and Camera	229 ms
When: Camera performs dolly-in (Standard Success Flow)	109 ms
Then: action returns true	3 ms
Then: Camera zooms in	2 ms
Then: Arthur reacts to the movement	102 ms
Then: Arthur's physical reaction is suppressed (hair stops)	2 ms
When: Camera is NOT recording	5 ms
When: Target Console is inactive	5 ms
When: Camera performs dolly-in multiple times	101 ms
When: Called with null arguments	9 ms

Успешный запуск тестов.

Результаты по тестированию:

В рамках задания была спроектирована доменная модель сцены на основе исходного текста и реализована с использованием подходов TDD + BDD.

Сначала были сформулированы тесты, описывающие ожидаемое поведение системы (сценарии, состояния, граничные случаи), после чего реализованы классы Arthur, Head, Console, Camera и сценарий performDollyIn. Реализация писалась итеративно: минимальный код для прохождения теста => рефакторинг => следующий тест.